

BÁO CÁO KỸ THUẬT

CCVA-HSRL-B6-02



MỤC LỤC

I. Giới thiệu chung.....	2
1. Thành viên.....	2
2. Kiến trúc tổng thể.....	2
3. Đường dẫn tới trang GitHub của dự án.....	2
II. Khả năng di chuyển & Thiết kế cơ khí.....	2
1. Bộ khung gầm.....	2
2. Hệ thống truyền động & Hệ thống lái.....	3
3. Thử nghiệm & Hệ bánh xe.....	5
III. Kiến trúc nguồn điện & Cảm biến.....	5
1. Sơ đồ nguồn điện & Các cổng kết nối phần cứng.....	5
2. Lựa chọn & Bố trí cảm biến.....	7
IV. Kiến trúc phần mềm & Chiến lược vượt chương ngại vật.....	11
1. Chiến lược sơ bộ.....	11
2. Thuật toán xử lý.....	11
3. Xử lý các trường ngoại lệ.....	13
V. Nhật ký thử nghiệm.....	14
1. Kiểm tra hệ số PID & Ngưỡng màu Threshold.....	14
2. Tiêu chí đánh giá.....	15
VI. Tư duy hệ thống & Quyết định kỹ thuật.....	15
1. Các quy định.....	16
2. Rủi ro có thể xuất hiện.....	16

I. Giới thiệu chung

1. Thành viên

CCVA-HSRL-B6-02 là một đội thi gồm 3 thành viên: Đào Đức Hiếu, Đặng Quang Tùng, Nguyễn Viết Quang Huy đến từ THPT Chuyên Chu Văn An và Hung Stem Robotics Lab.

2. Kiến trúc tổng thể

Từ yêu cầu từ đề thi WRO B6 2026, chúng em bắt đầu phân tích những công cụ cần sử dụng. Do bảng thi WRO B6 2026 có hai vòng thử thách, trong đó có một vòng thử thách mở không có chướng ngại vật và vòng thử thách vượt chướng ngại vật, cùng với đặc điểm về cấu trúc của sa bàn, chúng em đã lựa chọn bộ công cụ Matrix Robotics System - Future Innovators nhờ sự đa dạng về cảm biến và số lượng cổng kết nối lớn cùng sự linh hoạt về phương thức lập trình.

Tổng thể, robot xe tự hành của chúng em sử dụng 02 cảm biến laser thường dùng để đo khoảng cách, cùng với đó là 01 cảm biến màu sắc và 01 cảm biến camera. Ban đầu, chúng em sử dụng phần mềm ‘Matrixblock Mini R4’ để lập trình robot nhờ sự đơn giản và thuận tiện trong phương thức lập trình. Tuy nhiên, qua một thời gian sử dụng, nhằm tối ưu hiệu quả và tăng độ chính xác trong bài chạy, chúng em đã chuyển sang sử dụng phần mềm ‘Arduino IDE’ nhờ sự cho phép can thiệp sâu hơn vào lập trình robot của phần mềm này khi chạy chương trình của vòng thử thách ‘Vượt chướng ngại vật’.

3. Đường dẫn tới trang GitHub của dự án

[ddhhieu/CCVA-HSRL-B6-02](https://github.com/ddhhieu/CCVA-HSRL-B6-02)

II. Khả năng di chuyển & Thiết kế cơ khí

1. Bộ khung gầm

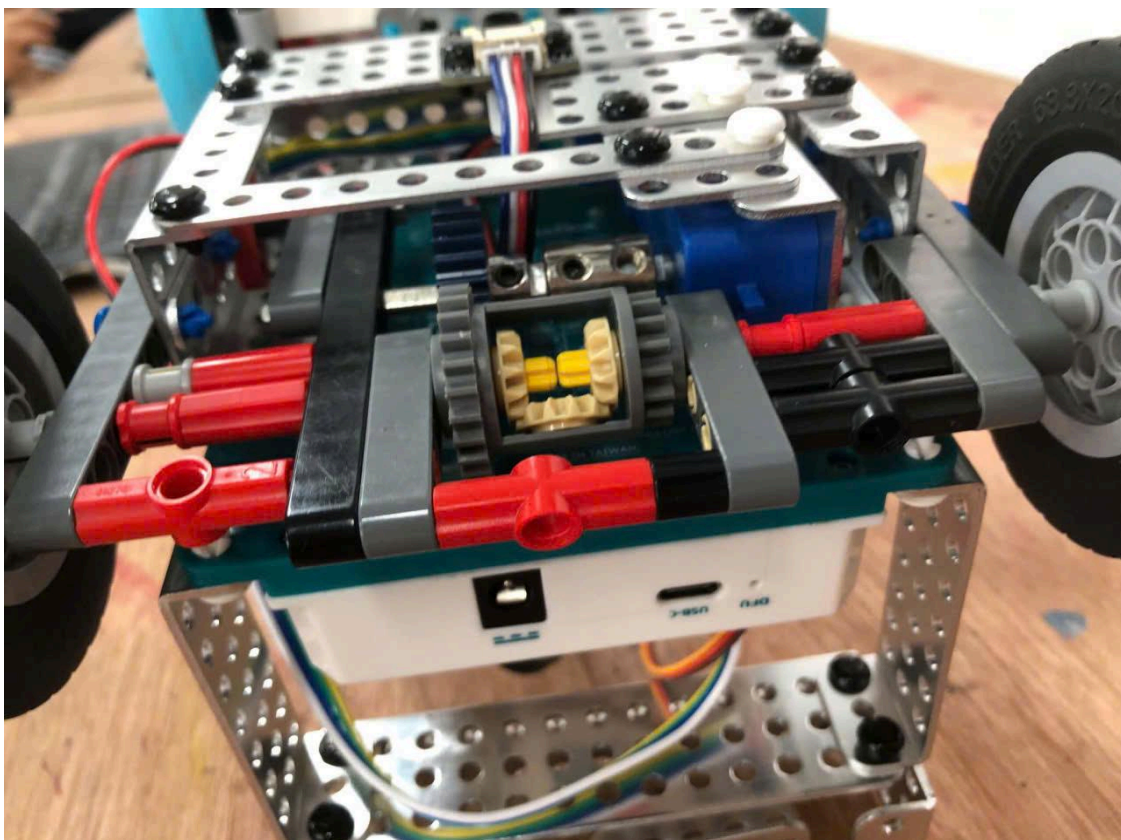
Bộ khung gầm robot được cấu tạo chính bởi 02 C shaped beam-9 Hole, 01 Flat Beam 9-Hole, 01 gusset plate 3x13, 02 gusset plate 3x7, 04 lego beam 5-hole, 02 lego beam 9-hole, 01 lego beam 11-hole, tất cả đều được kết nối bởi những thanh ‘Quick Connector’ của bộ công cụ Matrix. Nhờ đó, khung gầm có được sự chắc chắn của những thanh công cụ Matrix làm từ kim loại, đồng thời cũng rất linh hoạt thì tái tạo hệ truyền động nhờ sự linh hoạt của những thanh lego technic (Hình 2.1)



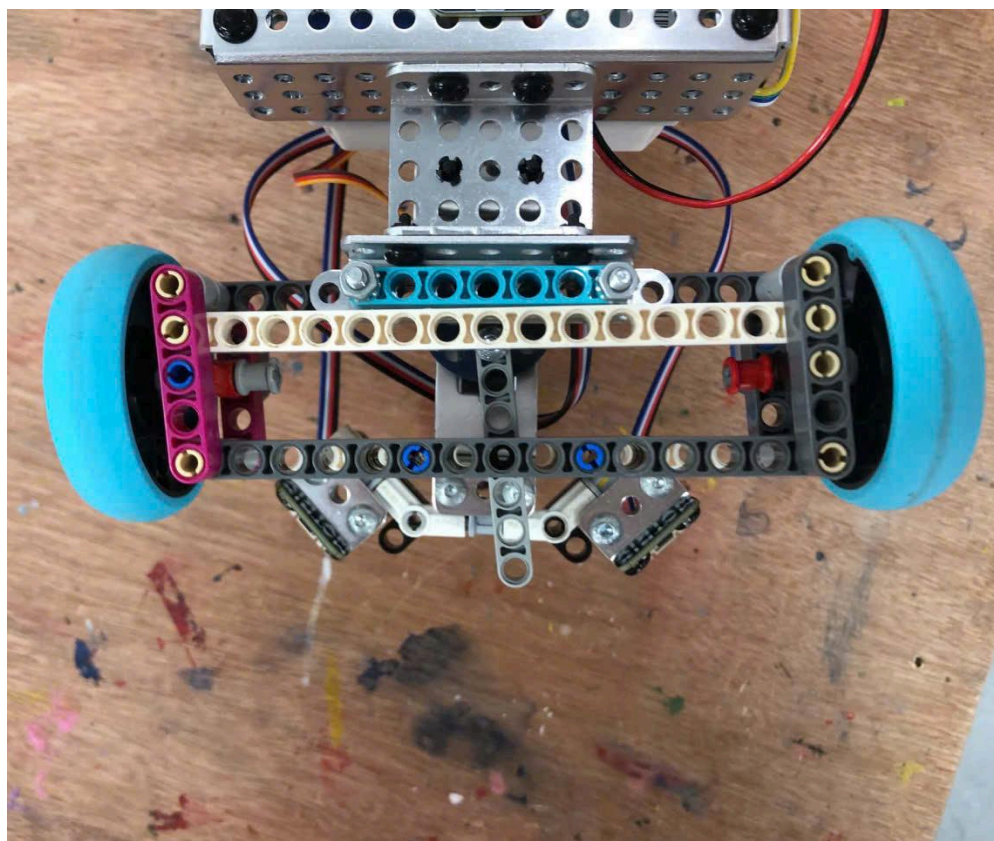
Hình 2.1

2. Hệ thống truyền động & Hệ thống lái

Để có thể truyền động một cách đồng đều và chính xác đến cả 2 bánh xe phía sau, xe được trang bị một bộ vi sai Technic gear differential 24-16 tooth. Khi tham gia băng đấu, từ yêu cầu mục 11.3 của BTC, với mong muốn chiếc xe sẽ di chuyển một cách mượt mà và chính xác nhất, chúng em sử dụng bộ truyền động dẫn động cầu sau với một động cơ motor ở phía sau kết nối với bộ vi sai thông qua ‘4mm axle collar with M4 set screw’ với 24 bánh răng (Hình 2.2). Do đó, tỉ số truyền của xe là 24:24 hay 1:1, tức với mỗi vòng quay của động cơ, bánh xe sau sẽ chuyển động một vòng. Bộ vi sai sẽ giúp cho khi chiếc xe rẽ hay vào cua, bánh xe bên phía hướng rẽ có thể dừng lại hoặc chạy chậm hơn bánh xe còn lại do nằm trên hai đường tròn đồng tâm khác bán kính. Hướng tới việc tối ưu sự cơ động, linh hoạt, gọn nhẹ cho xe, một bộ dẫn động rẽ hướng lái hình bình hành (parallelogram steering linkage) - Hình 2.3 - được cấu tạo từ những thanh lego technic đã được trang bị và được điều khiển bởi một động cơ servo được kết nối bởi một thanh lego beam 9-hole thay vì sử dụng hệ thống lái ‘Ackermann’ (bánh xe phía trong góc cua rẽ nhiều hơn bánh xe phía ngoài) như xe hơi trên thế giới.



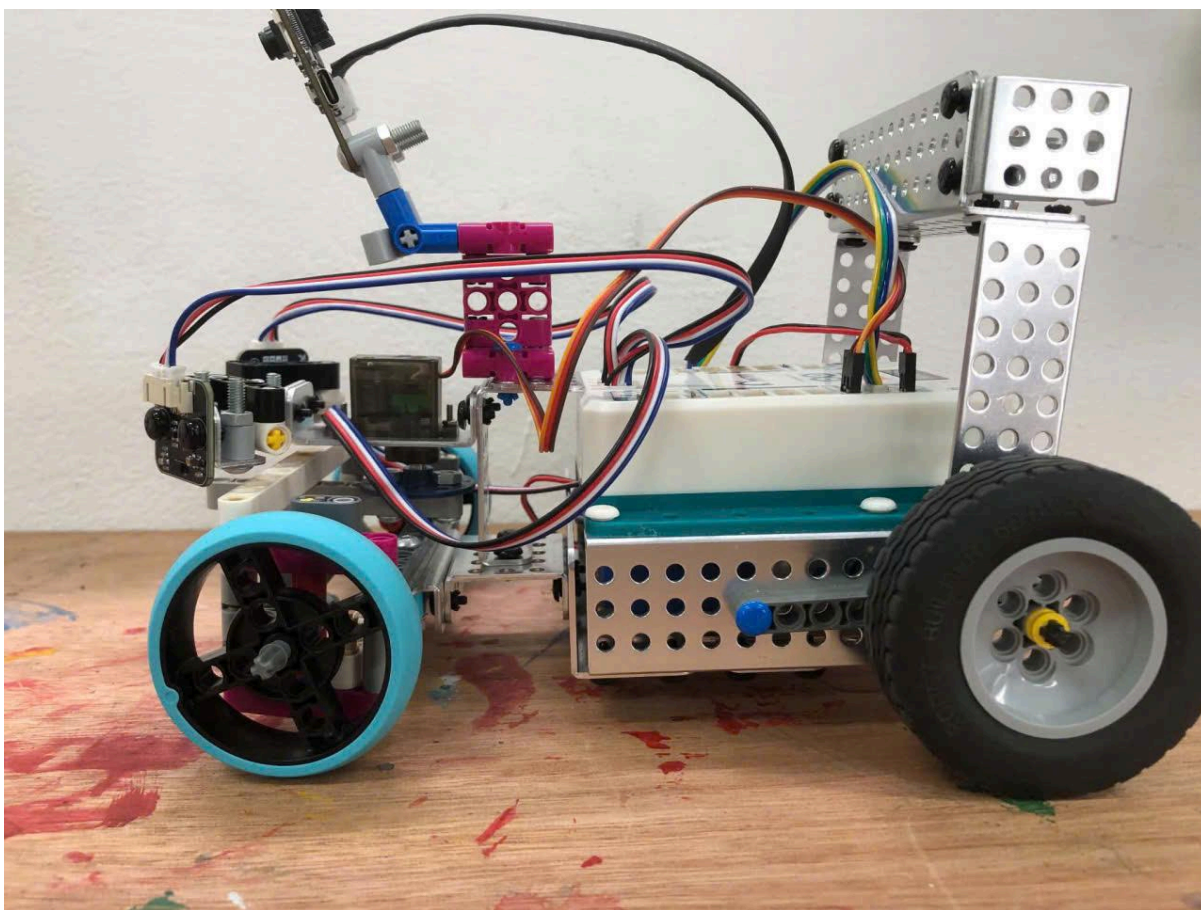
Hình 2.2



Hình 2.3

3. Thử nghiệm & Hệ bánh xe

Ban đầu, xe được trang bị bộ bánh trước với lốp ‘tire 56x26 balloon’ cùng bộ lốp sau ‘robot builder 68.8x20’. Tuy nhiên, sau thời gian chạy thử, do đặc tính khá “phồng” như cái tên balloon của bộ bánh trước cùng sự chênh lệch đến 6mm về bề ngang giữa lốp trước và lốp sau dù đường kính nhỏ hơn, xe khi vào cua chứng kiến sự lắc ngang tương đối và gây áp lực lên bộ rẽ hướng phía trước. Nếu sử dụng bộ lốp trước giống bộ lốp sau, xe sẽ bị “độn” thêm đáng kể về mặt kích thước, tuy sự chính xác có thể đảm bảo, nhưng khi bánh trước có kích thước lớn, sự ổn định khi rẽ sẽ không cao do sử dụng hệ thống rẽ hướng hình hình hành. Chúng em đã thay thành bộ bánh xe trước lego spike ‘wheel 56x14 technic’, tỉ lệ đường kính và bản mặt ngang giữa hai bộ bánh trước và sau khá tương đồng - hình 2.4 - sau khi được thay đổi nên xe vào cua chính xác và ổn định hơn đáng kể, đồng thời vẫn đủ bám khi xử lý trên những đoạn đường thẳng cần di chuyển tốc độ cao.



Hình 2.4

III. Kiến trúc nguồn điện & Cảm biến

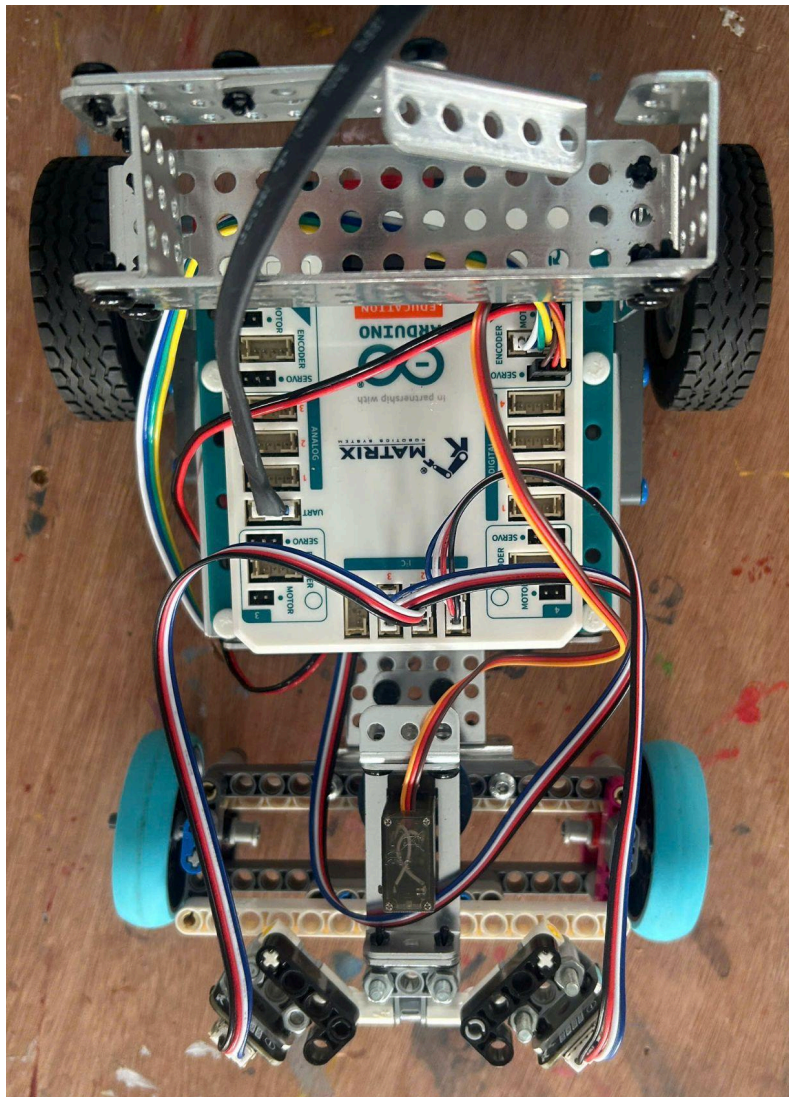
1. Sơ đồ nguồn điện & Các cổng kết nối phần cứng

Do sử dụng số lượng cảm biến khá lớn cùng 02 động cơ chính điều khiển toàn bộ hệ thống chuyển động, xe cần sử dụng 02 cell pin ‘ICR 18650 - 5000mAh -

3.7V' (Hình 3.1.1). Hai động cơ Encoder TT Motor và RC Servo lần lượt được cắm vào các cổng 'Motor', 'Encoder' và 'Servo' tại góc số 1 để đồng nhất trong việc lập trình và điều khiển chuyển động của xe. Hai cảm biến 'Laser Sensor V2' (MS-009 V2) được cắm vào hai cổng I2C1 và I2C2 nhằm thống nhất những nguồn dữ liệu input được trả về cho robot xử lý. Cùng với đó, một cảm biến 'Color Sensor V3' (MS-002V3)) được cắm vào cổng I2C3 và một cảm biến M-Vision Camera (MS-010) được cắm vào cổng UART (Hình 3.1.2)



Hình 3.1.1



Hình 3.1.2

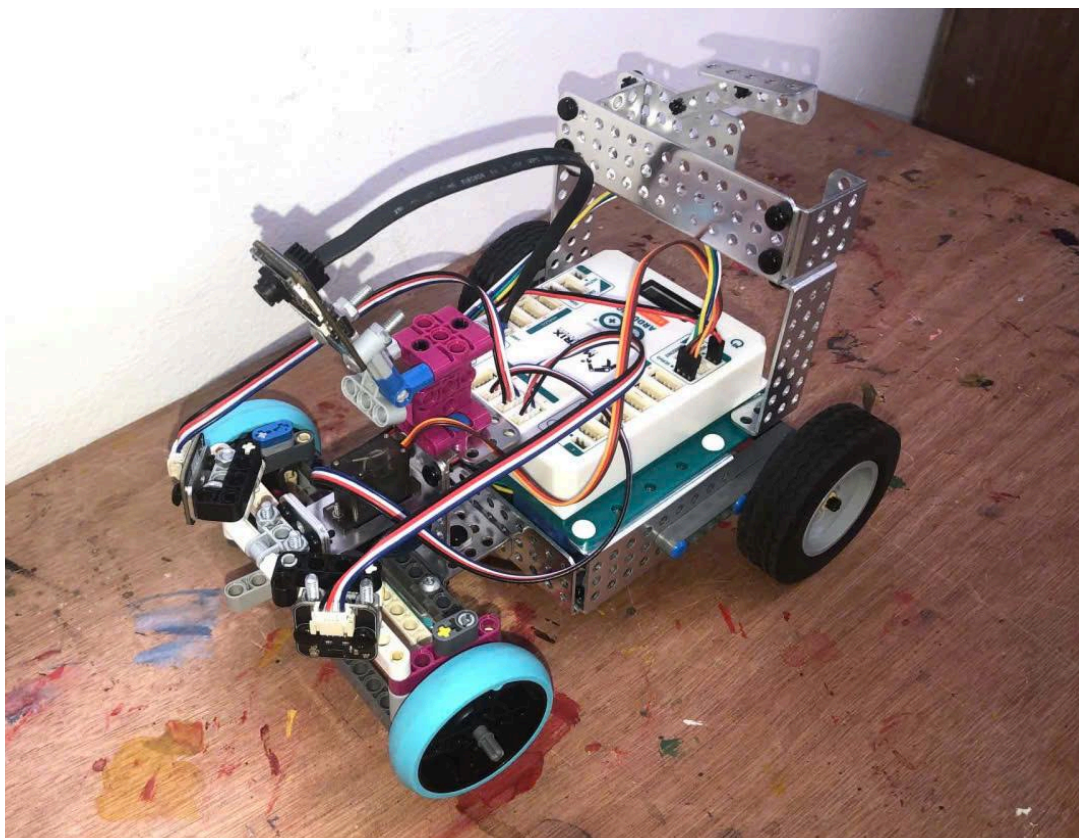
2. Lựa chọn & Bố trí cảm biến

Theo quy định số 11.11 của BTC, chúng em đã lựa chọn được những cảm biến như dưới đây:

Để xe có thể di chuyển bám tường, chúng em hiểu rằng 2 cảm biến ‘Laser Sensor V2’ (MS-009 V2) - hình 3.2.1 - cần được lắp đặt đối xứng và vuông góc với trục dọc của xe hoặc lệch một góc vừa phải để có thể đo khoảng cách giữa xe và tường. Khi di chuyển thực nghiệm, mỗi khi xe đến góc cua, khoảng cách giữa một cảm biến với tường sẽ giữ nguyên nhưng do tường bên trong đã không còn, cảm biến còn lại sẽ đo tường ở rất xa, lớn hơn đáng kể so với 2m - giới hạn đo của cảm biến. Điều đó tương đương rằng sẽ luôn có một cảm biến gặp lỗi dữ liệu trả về robot mỗi khi đến góc cua. Sau khi cân nhắc kỹ lưỡng các phương án đặt góc lệch, do đặc thù của sa bàn là một hình vuông khép kín với mỗi cạnh dài 3m, chúng em quyết định nối cảm biến với một thanh ‘Angle Element 1335 deg’ và đặt đầu còn lại của thanh đó vuông góc với trục dọc của robot, nhờ đó, cảm biến sẽ có một góc lệch khoảng 45 độ so với trục của robot (Hình 3.2.2). Khi di chuyển thực tế, nhờ góc nghiêng này, robot có thể vừa cân bằng đo được khoảng cách tường phía trước và tường ở hai bên của robot.



Hình 3.2.1

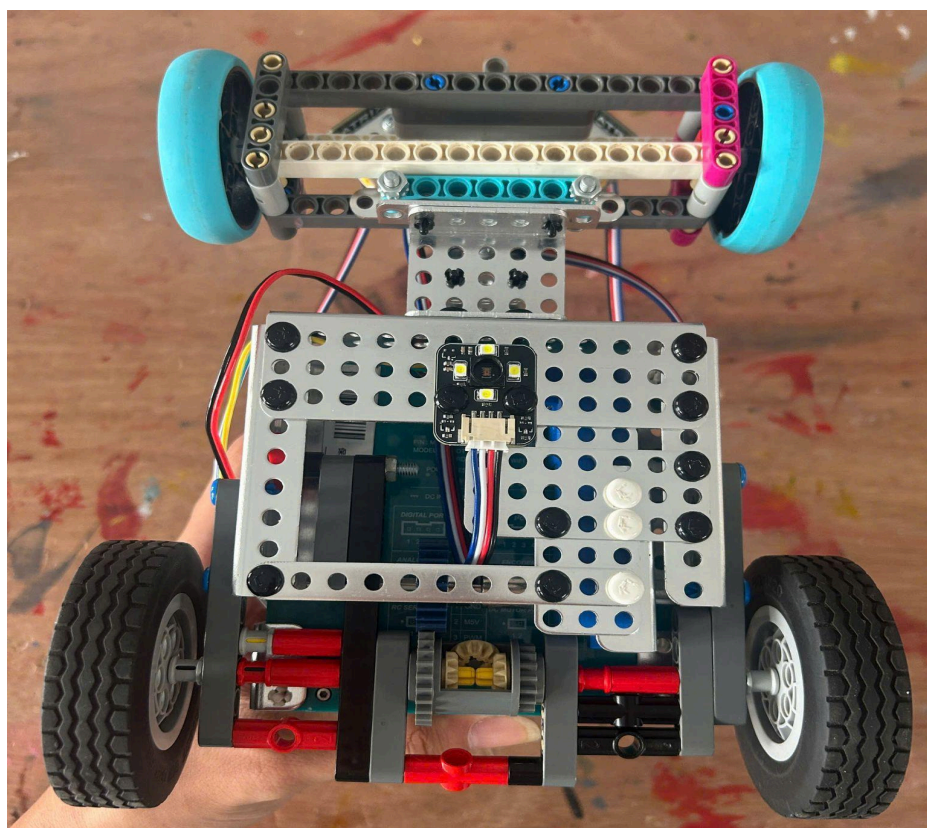


Hình 3.2.2

Về cảm biến ‘Color Sensor V3’ (MS-002V3) - hình 3.2.3, chúng em sử dụng với mục đích cho xe nhận biết được khi nào cần rẽ tại bốn góc của hình vuông sa bàn. Thông qua việc nhận biết được màu xanh hoặc cam của các vạch chéo tại bốn góc của, xe có thể biết được mình đang ở đâu và quyết định rẽ. Vì vậy, cảm biến màu sắc ‘Color Sensor V3’ (MS-002V3)) được đặt tại gầm xe, lệch cho với trung điểm xe khoảng 0.5cm về phía bánh sau (Hình 3.2.4). Vì khi rẽ, bánh xe trước đóng vai trò rẽ hướng, bánh sau sẽ gần như quay tại chỗ, đóng vai trò là tâm đường tròn của góc rẽ. Nếu đặt ngay tại trung điểm của xe, trong khi chủ yếu bánh trước của xe đóng vai trò rẽ, xe sau khi rẽ và đi bám tường có thể bị rung lắc giai đoạn đầu. Do đó, việc đặt cảm biến hơi lùi về sau sẽ giúp chúng em giảm thiểu một vài tính toán khi lập trình và những sai số khi xe vào góc cua.



Hình 3.2.3

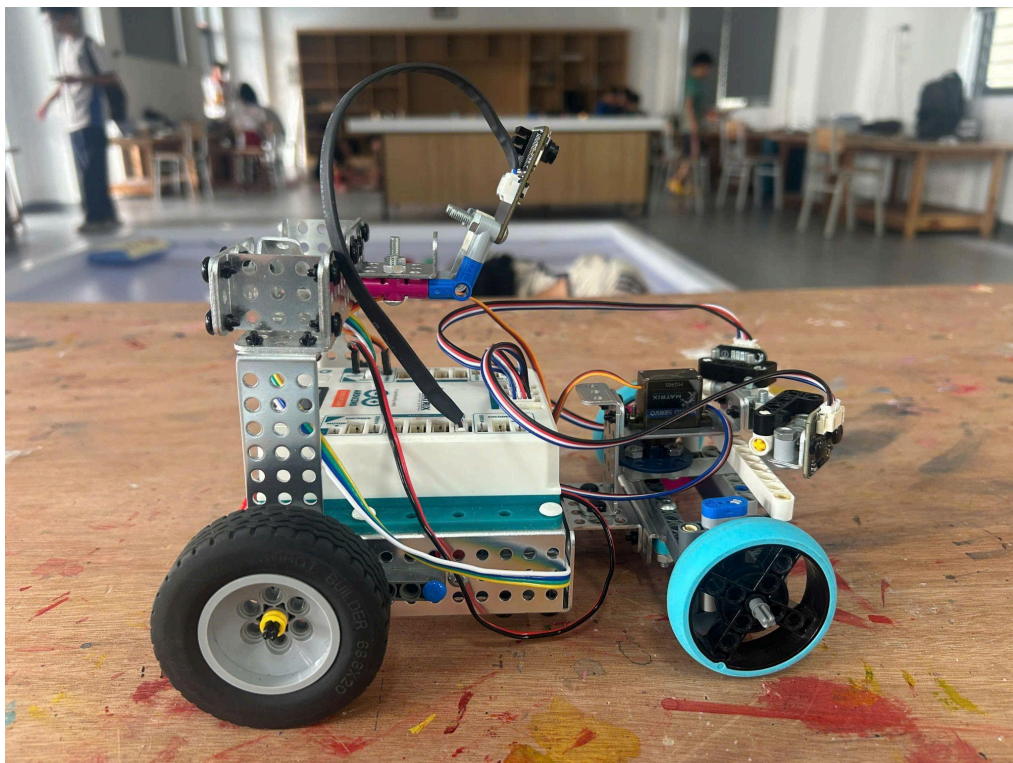


Hình 3.2.4

Để tránh được các đèn giao thông tại vòng thử thách ‘Vượt chướng ngại vật’, cảm biến M-Vision Camera (MS-010) - hình 3.2.5 - được chúng em sử dụng để nhận biết màu sắc của các cột đèn giao thông, nhờ đó xe có thể biết rằng xe sẽ cần rẽ hướng nào. Tuy nhiên, cảm biến M-Vision Camera (MS-010) cần một góc nhìn có thể bao quát toàn cảnh sa bàn nhằm nhận biết rõ màu sắc của mỗi khối đèn giao thông. Do đó, cảm biến này được đặt lệch về phía sau robot, cao hơn tâm của bộ bánh sau khoảng 15cm và nằm trước tâm của bộ bánh sau khoảng 10cm (Hình 3.2.6)



Hình 3.2.5



Hình 3.2.6

IV. Kiến trúc phần mềm & Chiến lược vượt chướng ngại vật

1. Chiến lược sơ bộ

- Vòng 1: khởi động => đi mù tới khi bắt được line rẽ => (bắt line rẽ => rẽ => bám tường) x11 => bắt line rẽ => rẽ => bám tường về khu vực xuất phát => kết thúc. ([CCVA-HSRL-B6-02 - WRO 2026 Future Engineers Open Challenge - YouTube](#))
- Vòng 2: Ra khỏi bãi đỗ => (không đọc được gì) bám theo tường 1 bên => [(đọc được màu) bám theo tọa độ khối và né tránh theo màu => thực hiện góc quay theo cảm biến màu và quét khối tiếp theo] x3 => về bãi đỗ xe. ([CCVA-HSRL-B6-02 - WRO 2026 Future Engineers Osbtacle Challenge - YouTube](#))

2. Thuật toán xử lý

- Thuật toán hệ số PID (Hình 4.1):
 - Robot sử dụng bộ điều khiển rẽ hướng bằng **PID** để duy trì khoảng cách ổn định với hai bên tường và điều khiển góc lái.
 - Thành phần **Kp** (proportional) giúp xe phản ứng nhanh khi xuất hiện sai lệch, **Ki** (Integral) trong bộ điều khiển **PID** giúp loại bỏ sai số tĩnh (steady-state error), giúp xe duy trì khoảng cách chính xác với tường khi chạy thẳng, còn **Kd** (derivative) giảm dao động và hiện tượng quá điều chỉnh.
 - Thuật toán này giúp robot di chuyển mượt, ổn định và phù hợp với yêu cầu chạy tốc độ cao xuyên suốt cả quá trình chạy thẳng và vào góc cua.

```
Error = (constrain(MiniR4.I2C1.MXLaserV2.getDistance(), 0, 800) - (mm * 1.53)) / 10;  
integral = integral + Error;  
Derivative = Error - LastError;  
PID = ((Kp * Error) + (Ki * integral)) + (Kd * Derivative);
```

Hình 4.1

- Thuật toán xử lý màu sắc của cảm biến M-Vision Camera (MS-010):
 - Cam có đèn fill light để đảm bảo ánh sáng chiếu đến vật và màu sắc vật thể trong các điều kiện khác nhau. Trong hàm ‘finding’, sử dụng hàm ‘find blobs’, ‘pixels threshold’ và ‘area threshold’, kết hợp với việc điều chỉnh threshold editor trên ‘OpenMV’ để lọc ra các đốm màu là mục tiêu dò theo (đọc màu đỏ, xanh,...) kết hợp với việc hạ ngưỡng độ phủ màu (density) xuống 50%, qua đó giúp giảm nhiễu, loạn khi đọc màu. Cam sẽ hạn chế đọc vật thể ở xa, ưu tiên các vật có tọa độ y lớn nhất (ở gần hơn), đồng thời dùng hàm ‘map’ để nhóm các ID đọc được khác nhau về cùng.

- Cơ chế ổn định: cho phép mất dấu tối đa 3 frames trước khi dừng robot, khi đột ngột không thấy trong 1-3 khung hình sẽ gửi vị trí cũ (last valid data) thay vì ngắt luôn, giúp chạy mượt mà. Về giao tiếp camera phải chờ lệnh 'G' từ đầu não robot mới chụp ảnh và lấy dữ liệu, bao gồm 4 thông số là 'màu, x, y, area'. nếu không có dữ liệu sẽ gửi gói dữ liệu id 255 để không bị 'timed out'.
- Thuật toán xử lý của não khi nhận dữ liệu từ cảm biến cảm biến M-Vision Camera (MS-010):
 - Khi bắt đầu, robot đi ra khỏi bãi đỗ xe sau khi được bấm nút chạy. sử dụng camera M-Vision, robot sẽ tìm và đọc các khối trụ, hay đèn giao thông.
 - Khi ở khoảng cách quá xa, cảm biến sẽ bỏ qua những khối đó (bằng cách gửi giá trị trống), và ưu tiên các khối ở gần. Khi đã khối đã ở một khoảng cách nhất định, robot sẽ tiến hành di chuyển, sử dụng các dữ liệu x, y được trả về từ camera để căn chỉnh sao cho khối nằm trên trục dọc robot, hay ở giữa camera. khi chuyển động đến gần, robot sẽ đọc màu và tiến hành rẽ trái và phải theo từng màu dựa theo dữ liệu camera.
 - Khi không đọc được dữ liệu gì, robot sẽ tiến hành đi bám theo tường ở một khoảng cách nhất định, dựa vào cảm biến Laser Sensor V2, đảm bảo chuyển động mượt mà và ổn định. Cảm biến màu Color Sensor V3 được sử dụng để đọc các vạch màu cam và xanh dương như các "checkpoint" khác nhau, qua dữ liệu đọc được sẽ làm các hành động và sử dụng thuật toán khác nhau.
 - Khi ở góc, robot sẽ bắt đầu quay dần dần để quét các khối ở gần, khi tìm thấy, robot sẽ lại bắt đầu bám theo khối và thực hiện né khối, nếu không đọc được thì tiếp tục bám tường đến khi tìm được. Ngoài ra, robot có thể lùi khi gặp vật cản hoặc bị kẹt để tiếp tục hành trình. Sau khi đọc được đủ các checkpoint (qua đủ 3 vòng), robot sẽ hoàn thành chương trình.
- Thuật toán bám tường trái - phải (Hình 4.2 và 4.3)
 - Với bên tường trái, robot sẽ sử dụng cảm biến Laser Sensor V2 bên trái đo liên tục để xác định khoảng cách với 2 bên tường, khi vào hàm 'void loop', số 'mm' mà robot cách tường sẽ được chỉ định và nhân với '1.53' - tỉ lệ giữa đường thẳng từ cảm biến đến tường với đường thẳng hình chiếu của robot với tường. Khi robot lệch sang trái hoặc phải, hệ số 'Error' sẽ âm hoặc dương và đưa ra quyết định cho xe rẽ phải hay trái. Tương tự cho bên phải với cảm biến Laser Sensor V2 còn lại.

```

void Di_line_trai_toc_do_n_cm_n_Kp_n_Ki_n_Kd_n_cach_tuong_n(float toc_do, float cm, float Kp, float Ki, float Kd, float mm) {
    MiniR4.M1.resetCounter();
    integral = 0;
    LastError = 0;
    while(!(abs(MiniR4.M1.getDegrees()) > ((cm / 20.5) * 360)))
    {
        MiniR4.M1.setSpeed(toc_do);
        Error = (constrain(MiniR4.I2C1.MXLaserV2.getDistance(), 0, 800) - (mm * 1.53)) / 10;
        integral = integral + Error;
        Derivative = Error - LastError;
        PID = ((Kp * Error) + (Ki * integral)) + (Kd * Derivative);
        MiniR4.RC1.setAngle(constrain((90 + PID), 60, 120));
        LastError = Error;
    }
    MiniR4.M1.setBrake(true);
}

```

Hình 4.2

```

void Di_line_phai_voi_toc_do_n_quang_duong_n_Kp_n_Ki_n_Kd_n_cach_tuong_n(float speed, float cm, float Kp, float Ki, float Kd, float mm) {
    MiniR4.M1.resetCounter();
    integral = 0;
    LastError = 0;
    while(!(abs(MiniR4.M1.getDegrees()) > ((cm / 20.5) * 360)))
    {
        MiniR4.M1.setSpeed(speed);
        Error = (constrain(MiniR4.I2C2.MXLaserV2.getDistance(), 0, 800) - (mm * 1.53)) / 10;
        integral = integral + Error;
        Derivative = Error - LastError;
        PID = ((Kp * Error) + (Ki * integral)) + (Kd * Derivative);
        MiniR4.RC1.setAngle(constrain((90 - PID), 54, 120));
        LastError = Error;
    }
    MiniR4.M1.setBrake(true);
}

```

Hình 4.3

3. Xử lý các trường ngoại lệ

- Robot không được vạch màu để rẽ hướng
 - Robot tiếp tục di chuyển và tự rẽ hướng theo chương trình bám tường và trừ đi lượt rẽ đó trong mục ‘repeat’
- Không phát hiện được cột
 - Robot tiếp tục bám tường bằng cảm biến laser.
 - Camera liên tục quét để tìm lại cột.
 - Khi nhận diện được cột, robot chuyển sang chế độ tránh chướng ngại vật.
- Camera bị nhiễu hoặc nhận diện sai
 - Chỉ chấp nhận kết quả khi màu hoặc đối tượng được nhận diện ổn định trong nhiều khung hình liên tiếp.
 - Nếu dữ liệu không ổn định, robot bỏ qua kết quả đó và tiếp tục quan sát để tránh quyết định sai.
- Cảm biến laser mất tín hiệu

- Nếu chỉ một cảm biến laser mất tín hiệu, robot chuyển sang bám tường bằng cảm biến còn lại.

V. Nhật ký thử nghiệm

1. Kiểm tra hệ số PID & Ngưỡng màu Threshold

- Kiểm tra và hiệu chuẩn hệ số PID

Kp	Ki	Kd	Đánh giá kết quả
4	0	1.5	Xe di chuyển tốt ở đoạn thẳng nhưng lắc ở đầu và cuối các khúc cua. (Loại)
2	0.02	2	Xe đi đã ổn định hơn ở các khúc rẽ nhưng bị lắc ở những đoạn thẳng. (Loại)
1.5	0.02	1	Xe di chuyển ổn định trên hầu hết cung đường, tuy nhiên thử nghiệm lặp lại không đồng đều giữa mỗi lần. (Loại)
1.2	0.02	0.8	Xe di chuyển ổn định và đồng đều xuyên suốt thử nghiệm (Chọn)

- Hiệu chuẩn và đánh giá ngưỡng màu Threshold

Để tối ưu hóa khả năng nhận diện của cảm biến M-Vision Camera (MS-010) trong điều kiện ánh sáng thực tế trên sa bàn thi đấu, nhóm đã thực hiện quy trình hiệu chuẩn và thử nghiệm hệ thống theo các bước sau:

- Cấu hình camera cố định: Trước khi thu thập dữ liệu màu, các thông số cơ bản như Cân bằng trắng (White Balance), Độ lợi (Gain) và Độ phơi sáng (Exposure) đều được cố định nhằm triệt tiêu sự biến động màu sắc do môi trường.
- Tiêu chí đánh giá bộ tham số: Nhóm tiến hành thử nghiệm nhiều dải ngưỡng màu khác nhau và đánh giá dựa trên 4 chỉ số cốt lõi:

1. Tỷ lệ phát hiện chính xác chương ngại vật màu xanh.

2. Khả năng chống nhiễu (hạn chế nhận nhầm do ánh sáng phản xạ hoặc bóng đổ).
3. Tầm nhận diện hiệu quả tối đa.
4. Độ ổn định của thuật toán khi xe vận hành ở tốc độ cao.

Sau khi thử nghiệm mỗi cấu hình 15 lần tại các ngưỡng khoảng cách, điều kiện ánh sáng, vị trí màu sắc khác nhau, chúng em đưa ra bảng đánh giá sau:

Bộ tham số	Đánh giá hiệu năng	Kết quả
(26, 18, -128, -15, 127, -128), (38, 4, 55, 127, -128, 52), (30, 6, 25, 115, -128, 31), (30, 8, 14, 128, -127, 127)	Phát hiện được vật thể nhưng nhiễu mạnh dưới ánh sáng cường độ cao.	Loại
(26, 18, -128, -15, 127, -128), (38, 4, 55, 127, -128, 52), (30, 6, 22, 127, -128, 31), (30, 8, -128, -14, 127, -128)	Nhận diện ổn định, tối ưu khoảng cách phát hiện, chống nhiễu tốt.	Chọn
(26, 18, 122, 30, -127, 127), (-33, -4, 55, 107, 128, -52), (30, 6, 22, 127, -128, 31), (30, 8, -128, -14, 127, -128)	Triệt tiêu nhiễu tốt nhưng thường xuyên bỏ sót vật cản ở cự ly xa.	Loại

2. Tiêu chí đánh giá

- Thời gian hoàn thành: từ 1 phút đến 2 phút mỗi lượt chạy (60s đến 120s)
- Sai số bám tường trong ngưỡng 0 mm - 10 mm với mỗi đoạn đường chạy thẳng.
- Số lượt hoàn thành vòng thi thứ nhất: 28/30 lượt (xấp xỉ 93.3%)
- Số lượt hoàn thành vòng thi thứ hai:

VI. Tư duy hệ thống & Quyết định kỹ thuật

1. Các quy định

Thông số	Giới hạn
Kích thước	Kích thước của xe không được vượt quá 300 mm x 200 mm và chiều cao không được vượt quá 300 mm.
Nguồn điện	Các đội có thể sử dụng bất kỳ loại pin nào mà họ lựa chọn – không có hạn chế nào về thương hiệu, chức năng hoặc số lượng pin được sử dụng.
Khối lượng	Trọng lượng của xe không được vượt quá 1,5 kg.
Kỹ thuật	Mục 11.1 đến 11.5 trong tài liệu Kỹ thuật tương lai (B6) - OneDrive do BTC cung cấp.
Thời gian	Thời gian lướt chạy mỗi vòng không quá 180s mỗi vòng.

2. Rủi ro có thể xuất hiện

Rủi ro	Giải pháp
Không đọc được vạch màu	Cho xe tự động rẽ theo quy tắc bám tường và trừ một lần lặp lại vòng lặp.
Bánh xe rung lắc khi rẽ	Đặt pin cao hơn xe ở phía sau để nâng trọng tâm xe khi rẽ, giảm áp lực lên bộ đánh lái của xe.